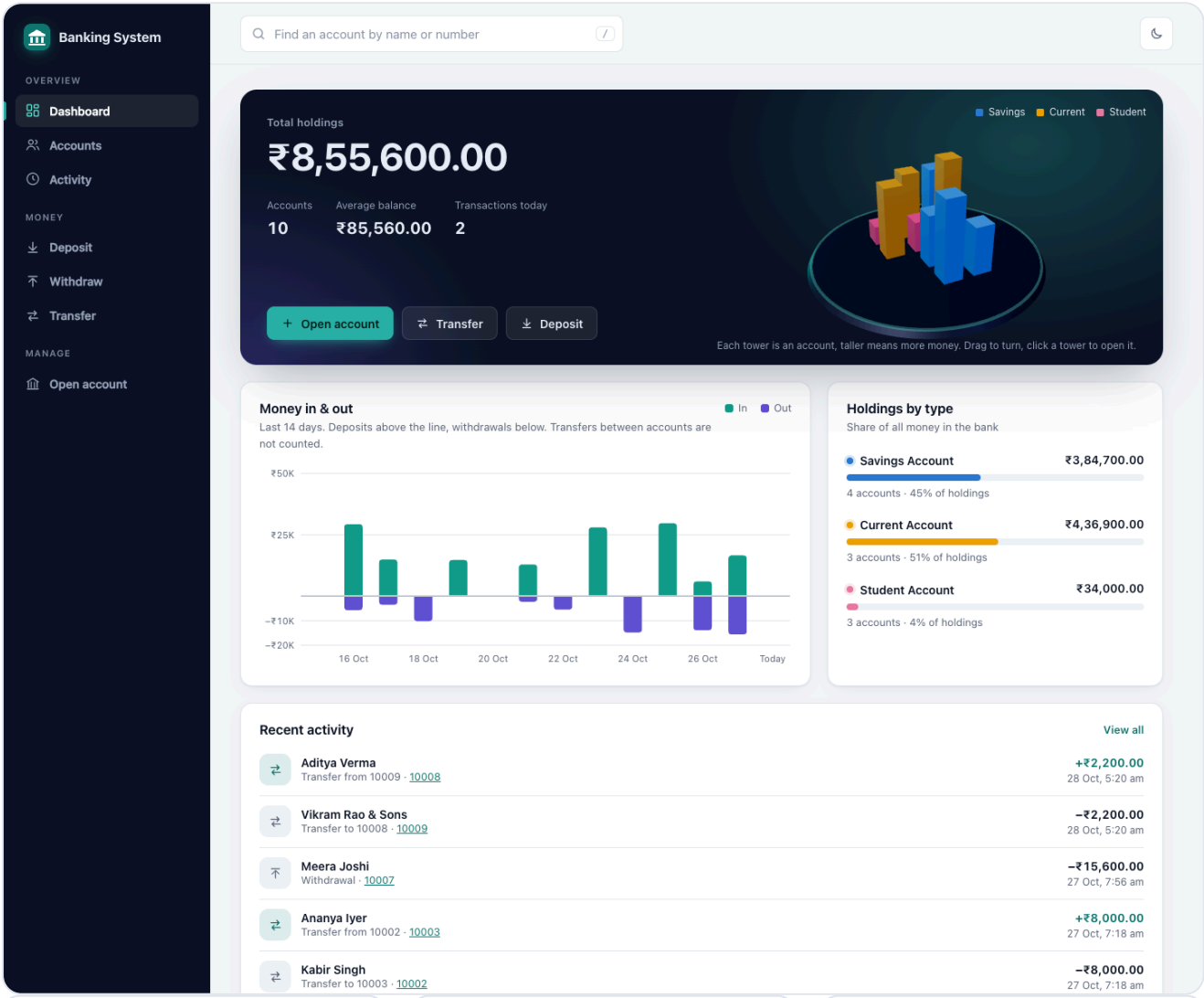
PROJECT REPORT

Banking System

A bank management website with a Java backend: accounts, deposits, withdrawals, transfers and a full transaction history.



LANGUAGE
Java 8+

LIVE AT
finvault-6ojq.onrender.com

DATE
October 2026

Contents

1 Introduction

2 Existing System

3 Proposed System

4 Feasibility Analysis

5 System Requirements

6 System Design

7 Modules

8 Business Rules and Validation

9 How to Run

10 Demo

11 Conclusion and Future Scope

1 Introduction

The main objective of this project is to develop a banking system that automates the everyday work of a bank branch. In a manual system every withdrawal or deposit needs paperwork, and finding the details of an account holder takes time. This bank management system stores all bank data in one place and gives staff a fast interface for retrieving customer details and recording transactions accurately.

The system lets staff add new customer accounts, deposit and withdraw money, transfer money between accounts and close accounts. Any account can be found in a few seconds by its number or the holder's name, and every transaction on every account is kept in a history that can be checked at any time. Access is protected by a login, so only authorised staff can use the system.

The system is a **website** served by a web server written in Java. It runs in any modern browser on a computer or a phone, and is hosted on the internet at **https://finvault-6ojq.onrender.com**.

2 Existing System

In the present system all banking work is done manually. Customers have to visit the bank for every withdrawal or deposit, and staff record each one in registers. This leads to several problems:

- Finding the account information of an account holder is slow.
- Calculations are done by hand, so mistakes in balances are common.
- Rules such as minimum balance or withdraw limits have to be checked by hand for every transaction.
- Registers take a lot of storage space and are hard to search.
- There is no quick overview of how much money the bank holds or what happened on a given day.

3 Proposed System

The proposed system is an automated bank management system. Through this software, staff can open accounts, deposit, withdraw, transfer money, close accounts and view the account list and the transaction history in very little time. The system checks every rule automatically and saves the data after every change. The proposed system has the following advantages:

- **User-friendly interface**: clear forms that show the account holder's name and available balance while the account number is typed.
- **Fast access to data**: accounts are kept in memory in a map and found instantly by account number.
- **Fewer errors**: balances are calculated by the program, and invalid amounts or rule violations are rejected with a clear message.
- **More storage capacity**: there is no fixed limit on the number of accounts.
- **Search facility**: search accounts by name, number or details, and filter transactions by kind.
- **Complete history**: every opening, deposit, withdrawal, transfer and closing is recorded with date, time and balance.
- **Look and feel**: a modern website with light and dark themes, an overview dashboard and 3D graphics, which also works on phones.
- **Quick transactions**: a deposit, withdrawal or transfer takes one short form.

All the manual difficulties in managing a bank are removed by computerising the work.

4 Feasibility Analysis

Not every idea is practical, so it is wise to study the feasibility of a problem before taking it up. A feasibility study looks at the impact the new system will have on the organisation. When the positives outweigh the negatives, the system is considered feasible. The study was done in three parts.

Technical feasibility

The system is technically feasible. It needs only the Java Development Kit, which is free and available for Windows, macOS and Linux. The website uses the web server built into the JDK and plain HTML, CSS and JavaScript, so no extra software or database server has to be installed or maintained. The 3D library (three.js) is included with the project, so the website also works without an internet connection.

Economical feasibility

Development of this application is highly economical. All the tools used are free and open source, and the system runs on the computers the organisation already has. After development, the organisation does not need to invest in licences or special hardware.

Operational feasibility

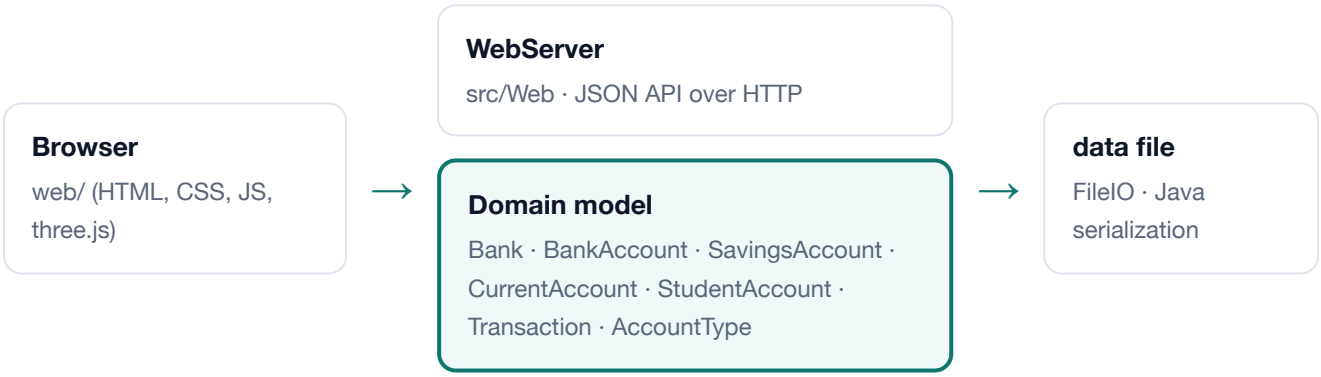
Staff need very little training. The website uses simple forms with one task each, explains its rules next to every form and shows clear error messages, for example when a withdrawal would go below the minimum balance.

5 System Requirements

ITEM	REQUIREMENT
Processor and memory	Any computer able to run Java 8 or newer; 2 GB RAM is enough
Operating system	Windows, macOS or Linux
Software	Java Development Kit 8 or newer
Browser (website)	A current version of Chrome, Edge, Firefox or Safari; WebGL for the 3D parts (optional)
Editor (optional)	Visual Studio Code with the Extension Pack for Java, which has ready-made run configurations

6 System Design

The system is split into layers. The browser shows the pages and talks to the **web server** through a small JSON API. The banking rules live in one place, the **domain model**, which has no user-interface code, and the model is saved to a data file after every change.



The browser reaches the domain model only through the WebServer, which checks the login on every request.

Main classes

CLASS	RESPONSIBILITY
`Bank`	Holds all accounts in a map keyed by account number, gives out new account numbers, performs deposits, withdrawals, transfers and closings, and records each one as a `Transaction`.
`BankAccount`	Abstract base class: account number, holder name, balance and minimum balance. Checks that amounts are valid and that a withdrawal keeps the minimum balance.
`SavingsAccount`	Adds a maximum amount per withdrawal and a 5% interest projection.
`StudentAccount`	A savings account for students with a low minimum balance, a fixed withdraw limit and the institution's name.
`CurrentAccount`	Business account with a trade licence number and no withdraw limit.
`Transaction`	One history entry: kind, account, amount, balance after and date and time.
`FileIO`	Loads the bank at start-up and saves it after every change. It writes to a temporary file first, so a failed save never destroys existing data.
`WebServer`	Serves the website files and the JSON API, and checks the login session on every request.

The class hierarchy uses inheritance (`StudentAccount` extends `SavingsAccount`, which extends `BankAccount`), polymorphism (each type overrides `withdraw` and `getDetails`) and encapsulation (balances can only change through the checked methods). Errors are reported with custom checked exceptions.

7 Modules

MODULE	DESCRIPTION
Login	Only staff with the correct username and password can use the system. The website keeps a secure session cookie after login.
Open account	Opens a Savings, Current or Student account with an opening deposit. The form changes to ask for the type-specific detail and shows the minimum balance.
Deposit	Adds money to an account. The website shows whose account it is before the money is deposited.
Withdraw	Takes money out of an account, checking the minimum balance and the withdraw limit. The website shows how much is available.
Transfer	Moves money from one account to another. The sender's rules apply, and both accounts get a history entry.
Close account	Closes an account after confirmation and pays out its balance. The account number is never given out again.
Account list	All accounts in a table that can be searched, sorted, filtered by type and exported to a CSV file.
Account page	One account's card, balance, rules, interest projection and full transaction history.
Activity	Every transaction in the bank, newest first, filterable by kind and searchable.
Dashboard	Total holdings, number of accounts, a chart of money in and out over the last 14 days, holdings by account type, recent activity, and a 3D view with one tower per account.

8 Business Rules and Validation

Account types

ACCOUNT TYPE	MINIMUM BALANCE	EXTRA DETAIL ASKED	MAX PER WITHDRAWAL
Savings	₹2,000	Maximum withdraw limit	The limit given when opening
Current	₹5,000	Trade licence number	No limit
Student	₹100	Institution name	₹20,000

Validation and error handling

SITUATION	EXCEPTION	MESSAGE SHOWN
Account number does not exist	`AccNotFound`	Account Not Found
Amount is zero, negative or not a number	`InvalidAmount`	Amount must be greater than zero
Withdrawal would go below the minimum balance	`MaxBalance`	Insufficient Balance
Withdrawal is over the account's limit	`MaxWithdraw`	Maximum Withdraw Limit Exceeded
Opening balance below the minimum, or a missing name or detail	`IllegalArgumentException`	A message naming the field

> All rules are checked on the server in the domain classes, so they cannot be skipped from the browser. A transfer is checked like a withdrawal from the sender, and no money moves unless the whole transfer is valid.

9 How to Run

Online: open `https://finvault-6ojq.onrender.com` and log in with the admin password set on the server.

In Visual Studio Code: open the project folder, choose **Banking Website** in the Run and Debug panel, and log in with username `admin` and password `admin`. The website opens in the browser by itself at `http://localhost:8080`.

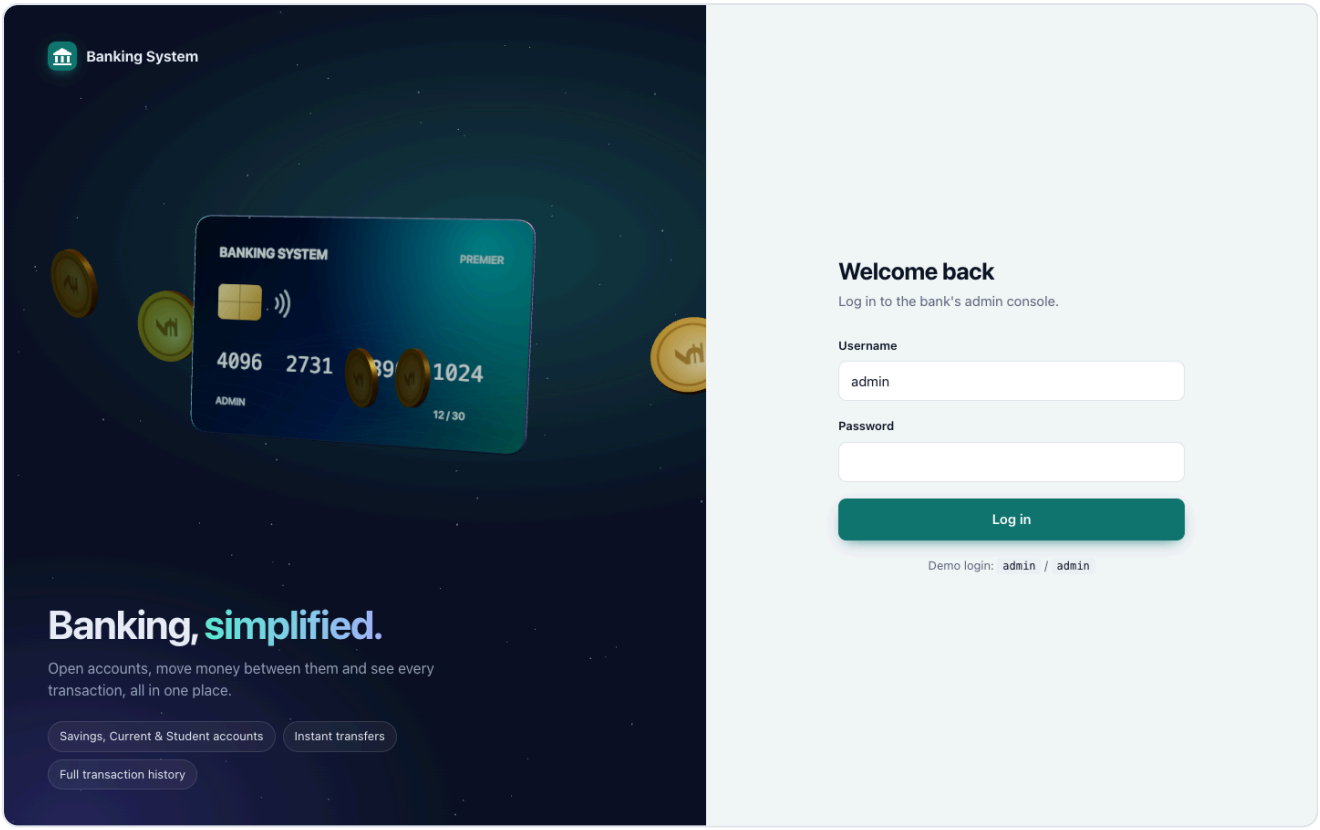
From a terminal, in the project folder:

`javac -d bin $(find src -name '*.java')`

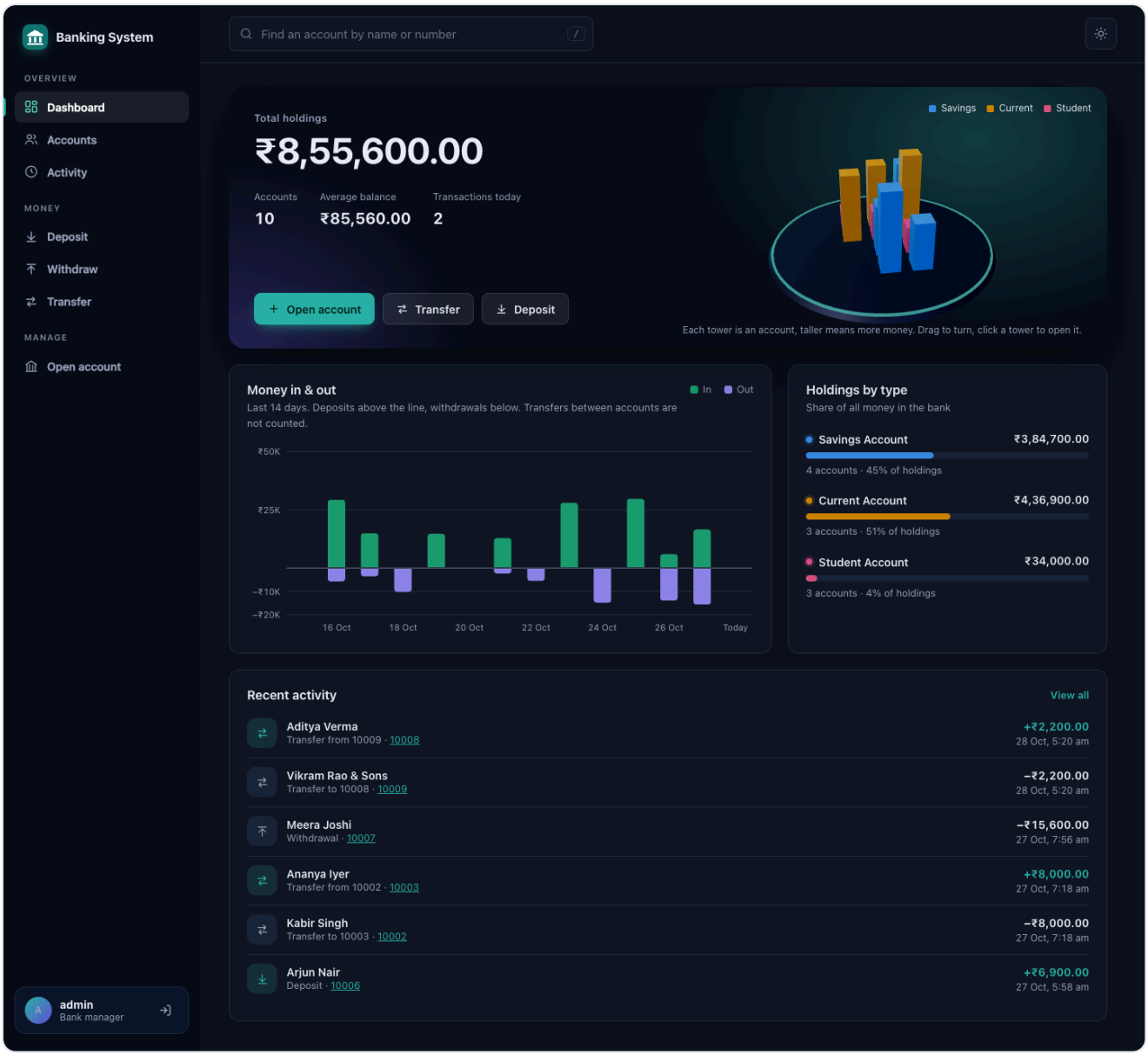
`java -cp bin WebApplication` starts the website.

> Always open the website through the Java server at http://localhost:8080. Opening the HTML file directly does not work, because the pages need the server's API.

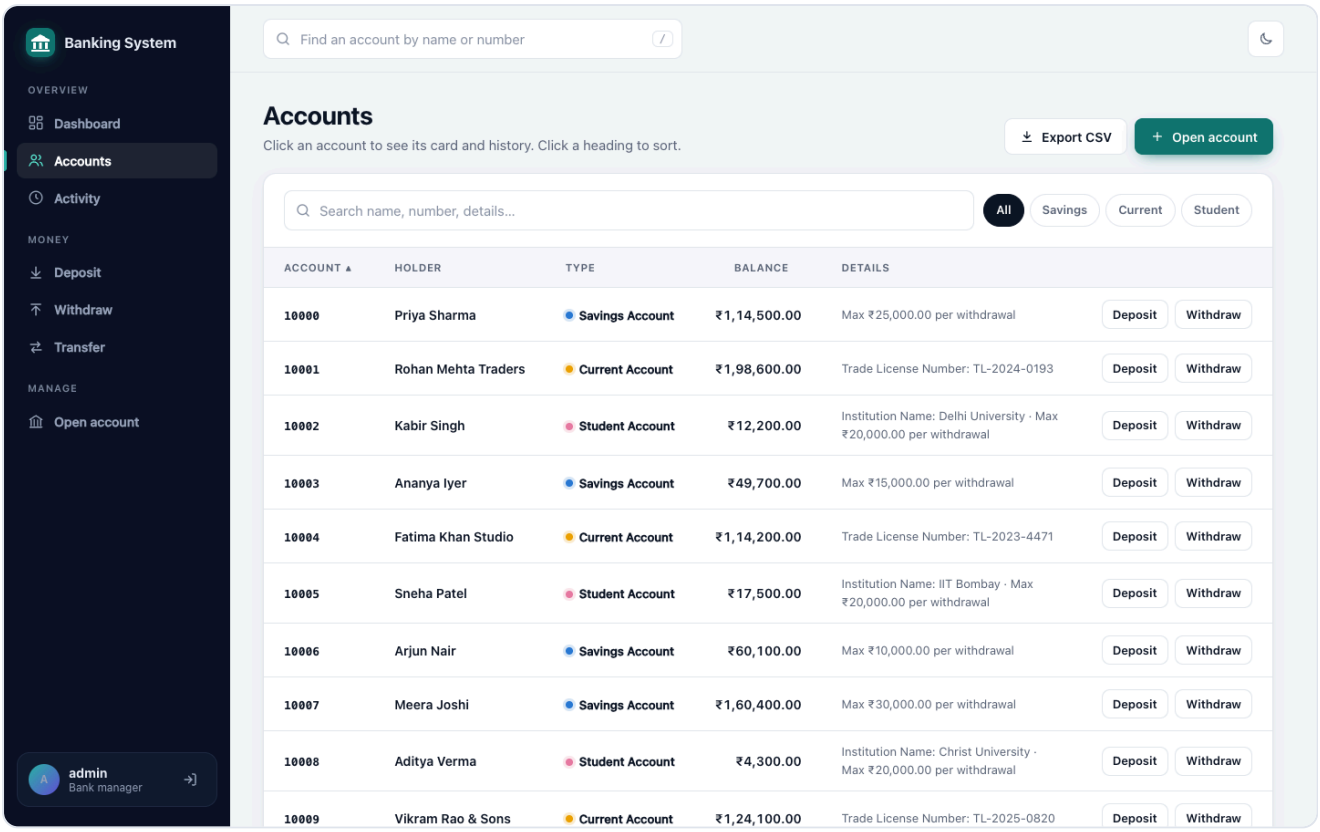
10 Demo



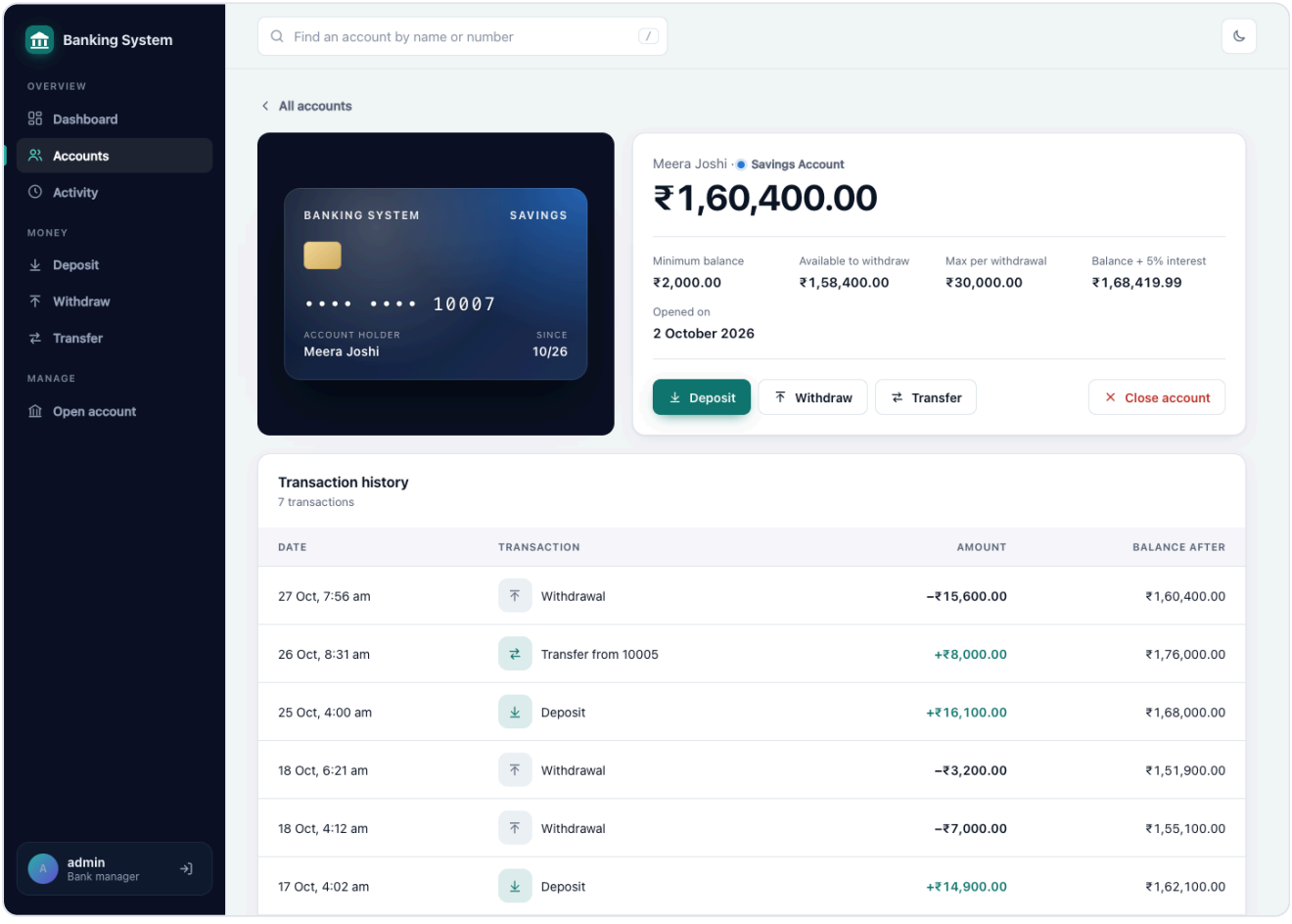
Login page with the animated 3D bank card



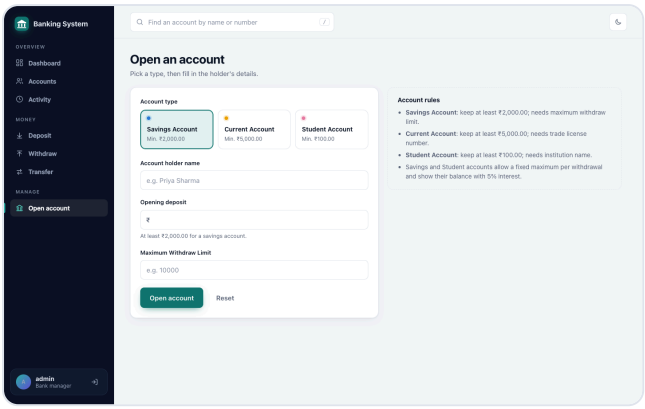
Dashboard (dark theme): totals, 3D account towers, money in and out, holdings by type, recent activity



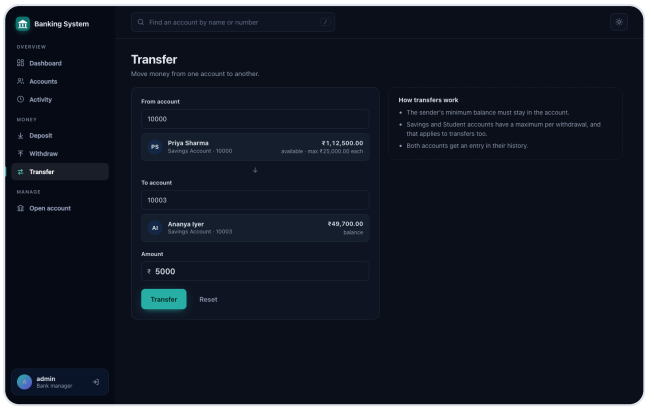
Account list with search, type filter, sorting and CSV export



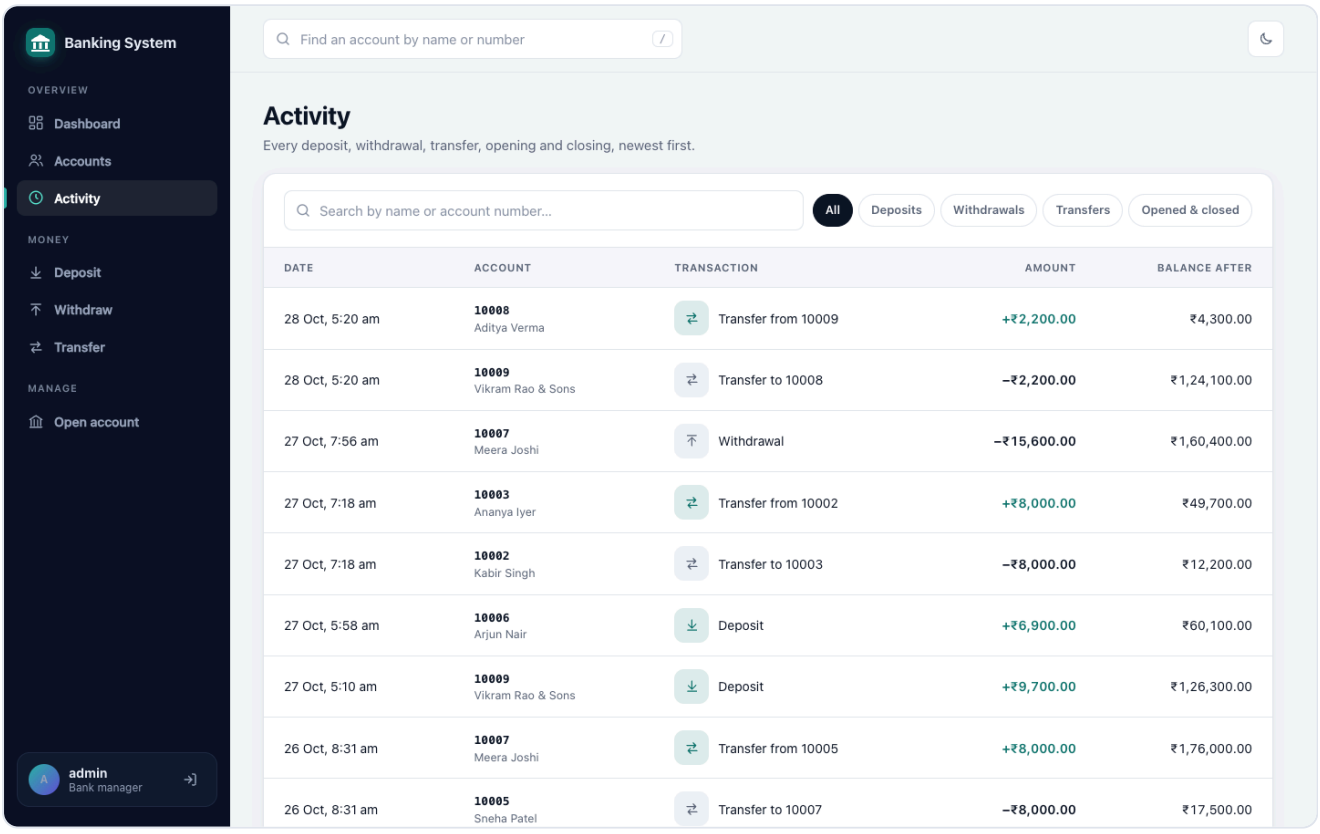
Account page: card, balance, rules, interest projection and transaction history



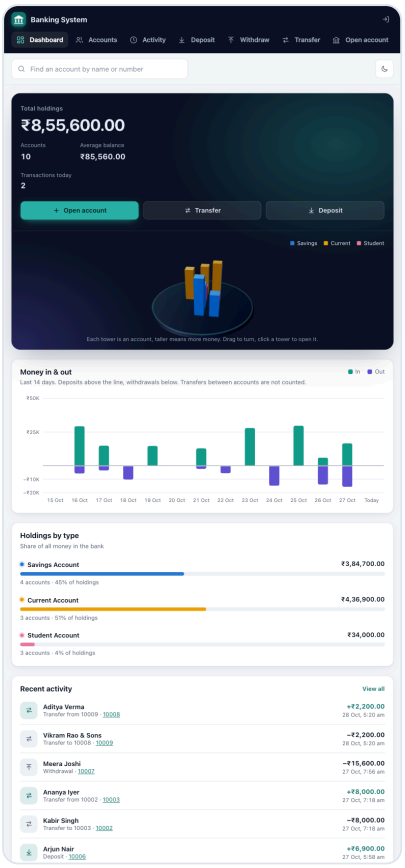
Opening an account



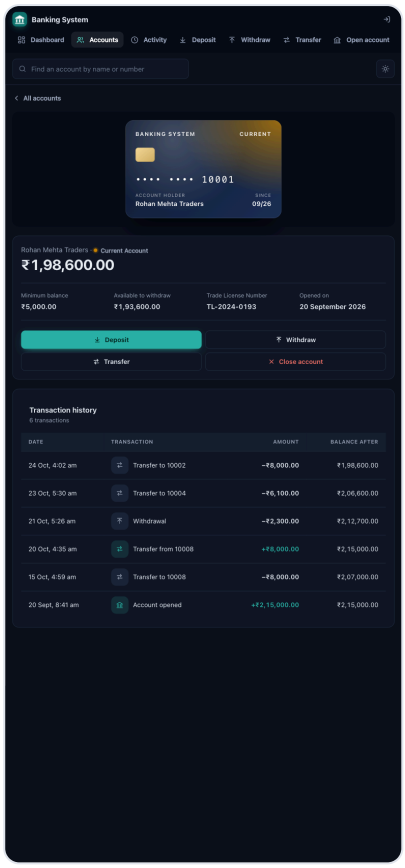
Transfer between two accounts



Activity: every transaction in the bank



Dashboard on a phone



Account page on a phone

The website screenshots use demo accounts.

11 Conclusion and Future Scope

The Banking System automates the main work of a bank branch: opening accounts, deposits, withdrawals, transfers and closings, with every rule checked by the program and every transaction recorded. Keeping the banking rules in a separate domain model keeps them in one place and easy to test, and the website adds an overview dashboard, a complete history and a modern interface that also works on phones. The website is deployed with Docker and is available on the internet.

Future scope

- Separate logins and roles for managers, cashiers and customers, with passwords stored as secure hashes.
- A database such as MySQL or PostgreSQL, so that accounts are kept when the server restarts.
- Automatic monthly interest on savings and student accounts.
- Printable account statements and PDF receipts for each transaction.
- Email or SMS alerts for large withdrawals.
- A custom domain name for the live website.